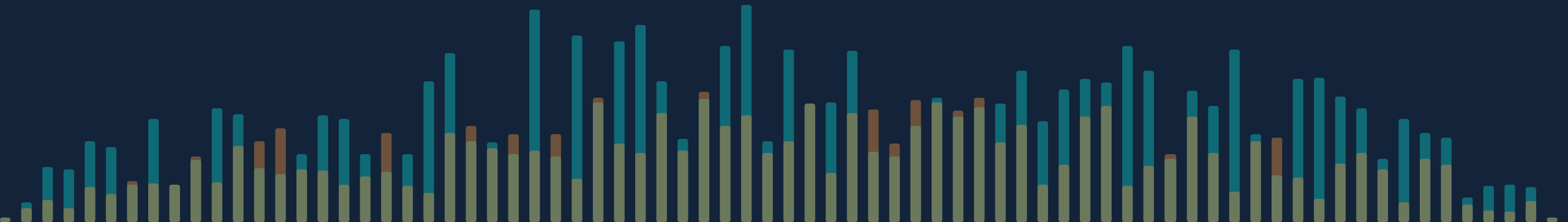
Speech Noise Reduction Using Long Short-Term Memory Networks

Skylar Denno
EECE 5644: Machine Learning
Northeastern University



PROBLEM STATEMENT

The problem

Noise is everywhere and hard for later systems to use



Learn the mapping from paired examples, so the model handles noise that changes over time.

Dynamic noise

The noise changes second to second.

Speech-like

The noise can be a voice or music. Can we tell which is the “correct” thing to focus on?

How powerful can we make the model? What about fast?

Goal: Make low-latency, real time model and a high-quality model

DATASET

Edinburgh Noisy Speech Database

11,572
clean / noisy
file pairs

28
speakers in
the train set



- Every clean file has a noisy twin.
- Organized by speakers
- These are recordings of short phrases with noise added over it

Source

University of Edinburgh DataShare
Centre for Speech Technology Research
doi.org/10.7488/ds/2117

Free for research use.

PREPROCESSING

Verification of data quality

- **unpaired** — no partner file
- **quiet** — RMS below −40 dBFS
- **clipping** — > 0.1 % samples at full scale
- **short** — under 1.0 second
- **bad_sr** — wrong sample rate
- **offset** — |DC offset| > 0.05

These are the conditions in which files would be dropped

Tested with broken files

8 broken pairs → skipped

2 silent files → quiet - skipped

2 square waves → clipping - skipped

2 short 44.1 kHz files → short, bad_sr - skipped

11,572

pairs kept · 0 dropped

MODEL SELECTION

Why an LSTM?

1

Noise is sequential

Nearby frames look alike. The last mask helps to predict the next.

2

Vanishing gradient

Hundreds of small gradients multiply down to zero in normal RNNs.

3

LSTM cell state adds gradients

The cell state adds instead of multiplies so frames from farther behind can effect the next one.

Framing

Many-to-many

One frame: 257 frequency bucket magnitudes. 257 gain values out.



DATASET

Encoding – Short-Time Fourier Transform

257
frequency bins
per STFT frame

32 ms
per frame
(512 samples @ 16 kHz)

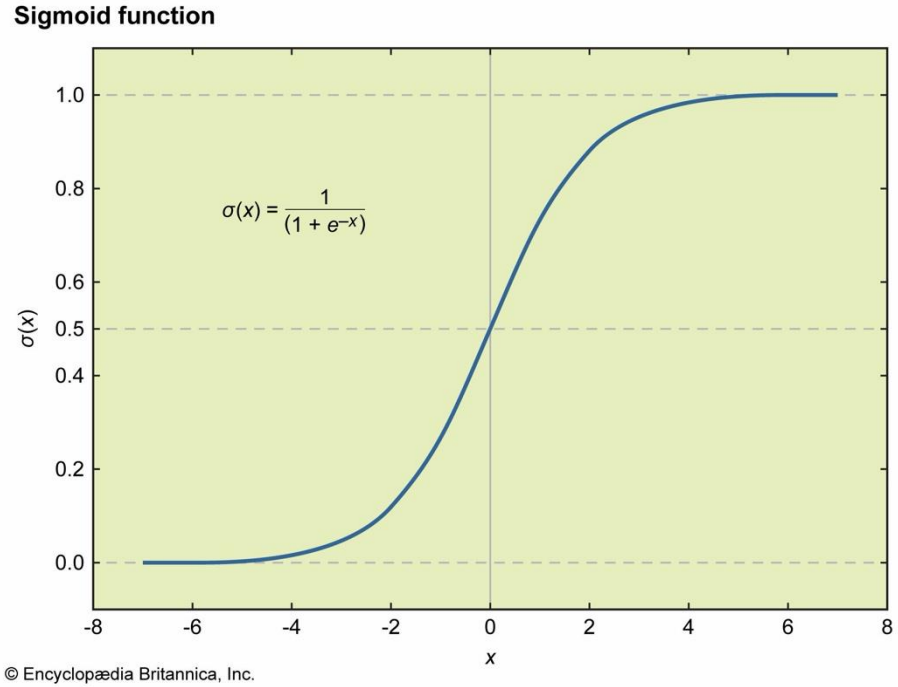
- Not just going to feed the raw audio digital samples into the LSTM as the training data
- Inherently a frequency problem (frequency bins)
- Also a time problem (time frames)
- So we use STFT which encodes both of these in a way that is easy for LSTM to learn from and easy for DSP processing

PIPELINE

Architecture summary

Noisy WAV — mono, 16 kHz → **STFT** — 257 × T complex → **log1p |Y|** — input features → **LSTM** — packed sequences → **Linear + Sigmoid** — 257 gain values → **Mask × |Y|** — clean magnitude → **iSTFT** — noisy phase

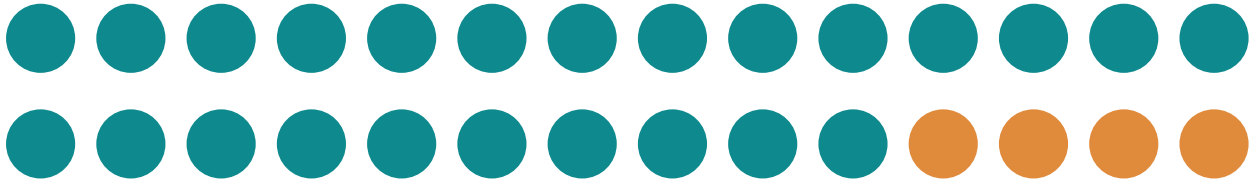
Input size	257 bins
Direction	Bidirectional (high quality) / Causal (standard quality) → 2 × hidden
Output head	Linear → Sigmoid, scaled by mask_max. Each freq. bin judged separately
Phase	Reuse the noisy phase



DATA SPLIT

Split by speaker, not by file

28 speakers, split before training



24 training speakers — 9,964 utterances

4 validation speakers — 1,608 utterances

Why it matters

A file split puts one voice in both sets. The model then might have an advantage by knowing the speaker. A speaker split stops this.

The randomizer is seeded for repeatability. The script ensures that the intersection of the training and validation speaker sets is empty.

VALIDATION

How to evaluate something qualitative like audio quality?

Optimization metrics

Validation loss / MSE

Squared error on speakers the model never saw in training.

R^2

How much spectral variance the model explains.

Which is more useful to validate on?

PESQ not differentiable :(
Also, the perceptual metrics might not always agree with each other. They measure different things.

Perceptual / signal metrics

PESQ

–0.5 → 4.5 Higher better

Perceptual Evaluation of Speech Quality

STOI

0 → 1 Higher better

Short-Time Objective Intelligibility

SI-SDR

dB – Higher better

Scale-Invariant Signal-to-Distortion Ratio

STAGE 4 — TRAINING

Training the network

Pad, then pack

Each batch pads to its longest file. Packing tells the LSTM where the real audio stops. A masked MSE stops the MSE from empty padding signal from being considered

Masked MSE loss

The loss ignores every padded frame, so silence adds no gradient.

What the model outputs

An IRM mask. Sigmoid gives a 0-1 gain per bin. The gain multiplies the noisy magnitude to estimate the clean one.

Settings · 4_train.yaml

Optimizer	`Adam`
Learning rate	`[Hyperparameter]`
Weight decay	`0.0`
Gradient clip	`1.0`
Hidden size / layers	`[Hyperparameter]`
Dropout	`[Hyperparameter]`
Batch size	`8`
Epochs	`64 — stop at overfit`
Checkpoint on	`validation MSE`

Run A 16 kHz · unidirectional · 512-sample frames · 25 epochs · batch 4

Run B 44.1 kHz · bidirectional · 2048-sample frames · 32 epochs · batch 8

TUNING

Hyperparameter tuning

1

Build the grid

List values in the config. Take every combination. 2^4 = 16

2

Short trials

Train each combination for 3 epochs, not 25 to see the general direction of improvement.

3

Best combination

Keep the lowest best validation MSE during the 3 epochs

4

Retrain in full

Run the winner for the full schedule.

Grid searched: 16 trials

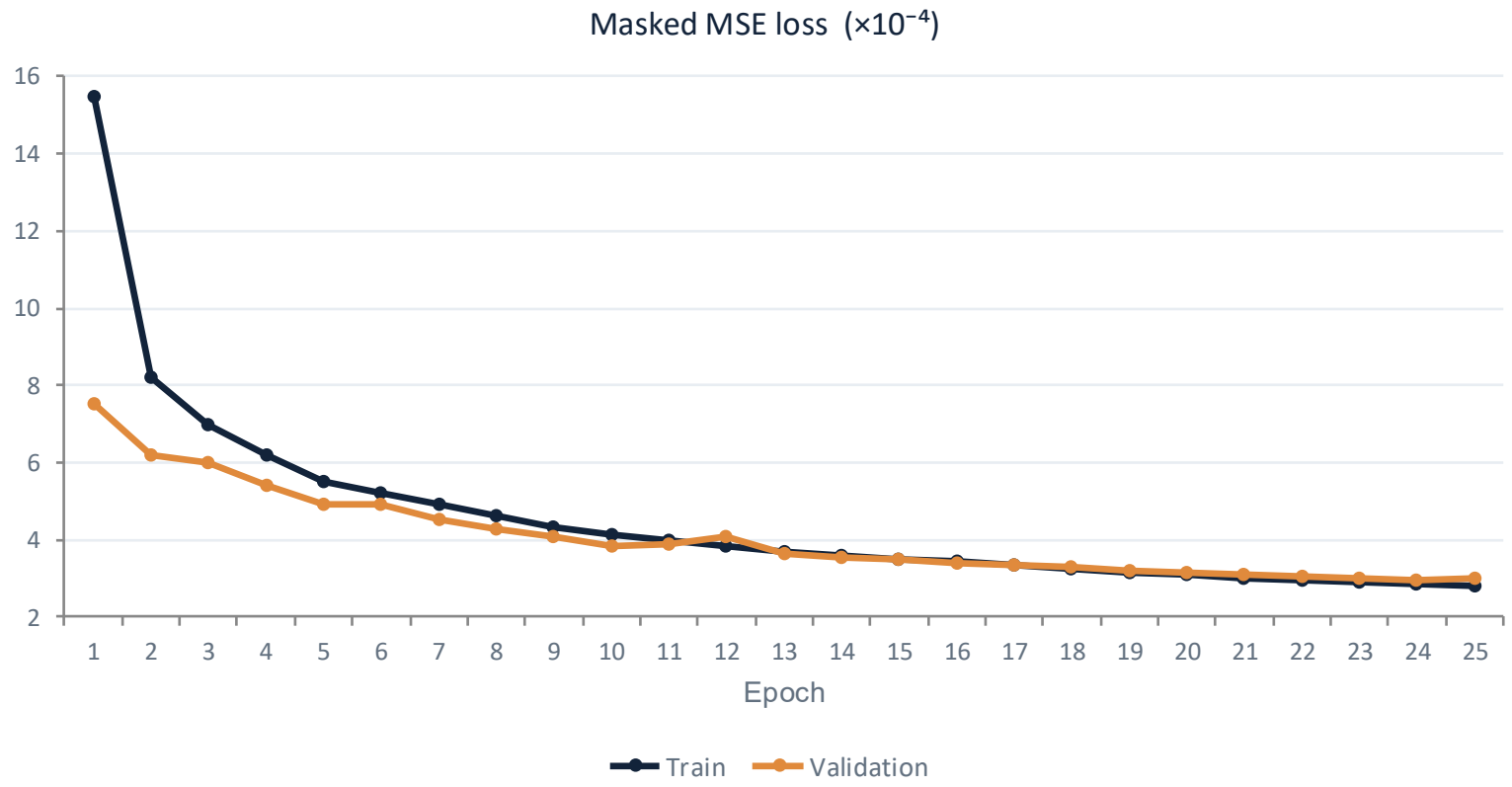
learning rate (Adam step)	0.001	0.0005
hidden size (capacity vs overfit)	128	256
layers (higher level relationship?)	1	2
Dropout (overfitting risk?)	0.0	0.2

Why use these four?

- Learning rate is very useful – gently change gradients or more aggressive?
- Hidden size helps to balance capacity and overfitting risk
- Layers help show if there is a higher level relationship between speech and noise
- Dropout finds if generalizing helps prevent overfitting

RESULTS: 16 KHZ RUN

Val loss learning curve



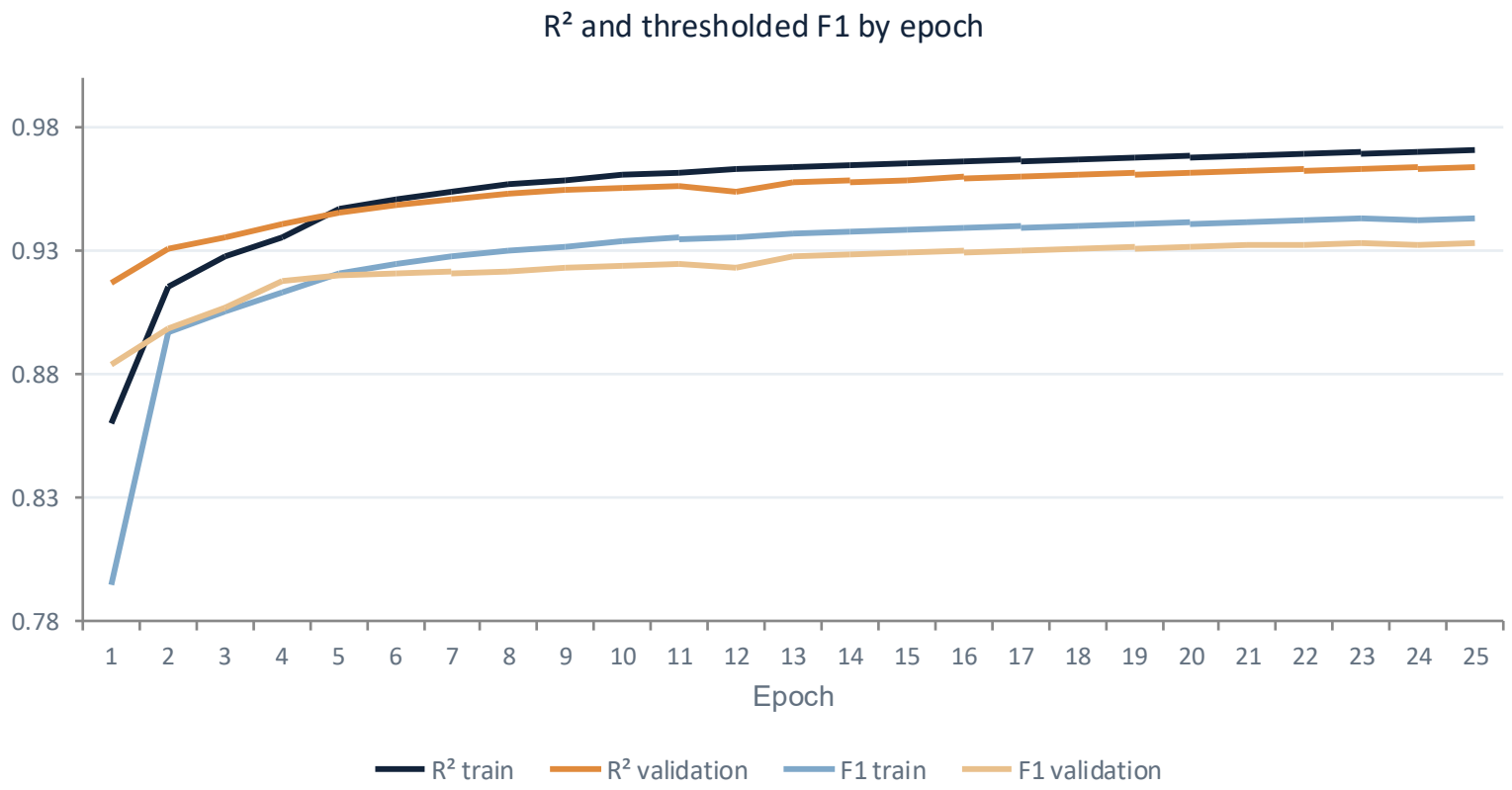
2.97×10^{-4}

best validation loss (epoch 24)

- Most of the gain is from first five epochs.
- Train and validation stay together. => No overfitting, even when compared to new speakers.
- After 25, overfitting starts happening and validation loss plateaus, increases, train loss keeps dropping.

RESULTS: 16 KHZ RUN

R^2 and F1 learning curves



0.964

best validation R^2 (epoch 24)

- R^2 above 0.96 on speakers it never saw.
- F1 comes from a threshold, not the model.
- Dropout is off during validation.

RESULTS: CONFUSION MATRICES

Confusion matrices: (But this is not a classification model)

Run A — 16 kHz unidirectional

validation, epoch 25

	Pred neg	Pred pos
Actual neg	**162,501,813**	3,283
Actual pos	6,739	**71,505**

Recall on active bins: 0.914 · missed: 0.086

Run B — 44.1 kHz bidirectional

validation, epoch 32

	Pred neg	Pred pos
Actual neg	**447,565,279**	1,897
Actual pos	3,377	**43,872**

Recall on active bins: 0.929 · missed: 0.071

Which frequency bins were correctly identified as noise and removed (gain < 0.5)? And which were identified correctly as speech and kept (gain >= 0.5)?

COMPARING THE TWO RUNS

Real time or best quality

Run A · Unidirectional

Past frames only. (Runs live)

Sample rate	**16 kHz**
LSTM direction	**Unidirectional (causal)**
Frame size	**512 samples**
Epochs / batch size	**25 / 4**
Positive-class recall	**0.914**
Best use	**Live calls, low latency**

Run B · Bidirectional

Needs the whole file first.

Sample rate	**44.1 kHz**
LSTM direction	**Bidirectional**
Frame size	**2048 samples**
Epochs / batch size	**32 / 8**
Positive-class recall	**0.929**
Best use	**Offline restoration**

TAKEAWAYS

The model cleans noisy speech recordings

1. Time context makes the mask smoother and artifact free.
   Bidirectional context helps increase the quality of output and reduce loss further.

2. A speaker-level split keeps the validation accurate to real performance.

3. The full pipeline (encoding) matters as much as the network.

Future work

- Complex masks and complex STFT encoding
- Complex-valued RNNs on cSTFT data
- Validate using speech quality perception metrics instead of validation loss?
- Do any other STFT parameters improve the model?
- More epochs by using more data, to make overfitting happen later?
- Bigger hyperparameter matrix
- Does the denoising help with things like automated speech recognization models? Do those models need different denoising parameters?



DEMO TIME!

Skylar Denno github.com/skylar-slideshows/ML_denoise

